Course_Card_Framer_1

https://gemini.google.com/app/5d169fb128b261a7

User prompt: I need your help to me with creation of this course card in Framer with hardcoded object : const course = { courseName: "How To YouTube", description: "...", mainCategory: "Content Creation", pricePaise: 199900, refundable: true }; can you walk me with steps? we will go step by step instead of everything in one go

Response: Step 1: Create the Code Component & Define the Data

First, let's create a new code component in Framer and define your data object with helper formatting for the price.

1. Open your Framer project.
2. In the left sidebar, navigate to the **Assets** tab (or press `Alt + 2`).
3. Click the **+** icon next to **Code** and select **New Component**. Name it `CourseCard.tsx`.
4. Replace the default template with the following starting skeleton:

```
import * as React from "react"

const course = {
    courseName: "How To YouTube",
    description: "Learn how to build, grow and monetize your channel.",
    mainCategory: "Content Creation",
    pricePaise: 199900,
    refundable: true,
}

// Helper to convert paise to formatted INR string (e.g., 199900 -> ₹1,999)
const formattedPrice = new Intl.NumberFormat("en-IN", {
    style: "currency",
    currency: "INR",
    maximumFractionDigits: 0,
}).format(course.pricePaise / 100)

export default function CourseCard() {
    return (
        <div style={{ color: "#ffffff", fontFamily: "sans-serif" }}>
            {/* Card contents will go here */}
        </div>
    )
}
```

Once you have this file set up, let me know and we'll move to **Step 2: Structuring the Card Layout & Border Styling**.

User prompt: it's done. lets move for next one.

Response: Step 2: Structuring the Card Layout & Border Styling

Now let's build the layout inside the component to match the visual structure from your wireframe (dark theme, dashed border, vertical stacking, and proper spacing).

Update your `CourseCard.tsx` file with the following code:

```
import * as React from "react"

const course = {
    courseName: "How To YouTube",
    description: "Learn how to build, grow and monetize your channel.",
    mainCategory: "Content Creation",
    pricePaise: 199900,
    refundable: true,
}

const formattedPrice = new Intl.NumberFormat("en-IN", {
    style: "currency",
    currency: "INR",
    maximumFractionDigits: 0,
}).format(course.pricePaise / 100)

export default function CourseCard() {
    return (
        <div
            style={{
                backgroundColor: "#1a1a1a",
                border: "1px dashed #ffffff",
                borderRadius: "12px",
```

```
                padding: "24px",
                display: "flex",
                flexDirection: "column",
                gap: "16px",
                color: "#ffffff",
                fontFamily: "sans-serif",
                maxWidth: "320px",
            }}
        >
            {/* Main Category */}
            <div style={{ fontSize: "14px", opacity: 0.8 }}>
                {course.mainCategory}
            </div>

            {/* Course Name */}
            <div style={{ fontSize: "20px", fontWeight: "bold" }}>
                {course.courseName}
            </div>

            {/* Description */}
            <div style={{ fontSize: "14px", opacity: 0.9, lineHeight: "1.4" }}>
                {course.description}
            </div>

            {/* Price */}
            <div style={{ fontSize: "22px", fontWeight: "600", marginTop: "8px" }}>
                {formattedPrice}
            </div>

            {/* Refundable */}
            <div style={{ fontSize: "14px", display: "flex", alignItems: "center", gap: "6px", opacity: 0.9 }}>
                <span>✓</span> Refundable
            </div>
        </div>
    )
}
```

Take a look at how it renders in your Framer canvas. Are you ready to move on to **Step 3: Adding Framer Component Controls (Props)** so you can easily edit these values directly from the Framer sidebar instead of hardcoding them?

User prompt: Let's move for next step. I attached of current image

Response: Step 3: Add Framer Property Controls

Now we'll expose the course data as **Framer Property Controls**. This allows you (or anyone editing the canvas) to click the card and change the course title, price, category, description, or toggle the "Refundable" badge directly from Framer's right-hand settings panel—without touching the code again.

Update your `CourseCard.tsx` file to the following:

```
import * as React from "react"
import { addPropertyControls, ControlType } from "framer"

interface CourseCardProps {
    courseName: string
    description: string
    mainCategory: string
    pricePaise: number
    refundable: boolean
}

export default function CourseCard({
    courseName,
    description,
    mainCategory,
    pricePaise,
    refundable,
}: CourseCardProps) {
    // Format paise to INR (₹)
    const formattedPrice = new Intl.NumberFormat("en-IN", {
        style: "currency",
        currency: "INR",
        maximumFractionDigits: 0,
    }).format(pricePaise / 100)

    return (
        <div
            style={{
```

```
            backgroundColor: "#1a1a1a",
            border: "1px dashed rgba(255, 255, 255, 0.3)",
            borderRadius: "12px",
            padding: "24px",
            display: "flex",
            flexDirection: "column",
            gap: "14px",
            color: "#ffffff",
            fontFamily: "Inter, sans-serif",
            width: "100%",
            maxWidth: "320px",
            boxSizing: "border-box",
        }}
    >
        {/* Category */}
        <span
            style={{
                fontSize: "13px",
                fontWeight: 500,
                color: "rgba(255, 255, 255, 0.6)",
                letterSpacing: "0.02em",
            }}
        >
            {mainCategory}
        </span>

        {/* Course Title */}
        <h3
            style={{
                fontSize: "20px",
                fontWeight: 700,
                margin: 0,
                lineHeight: "1.2",
                color: "#ffffff",
            }}
        >
            {courseName}
        </h3>

        {/* Description */}
        <p
            style={{
                fontSize: "14px",
                lineHeight: "1.5",
                margin: 0,
                color: "rgba(255, 255, 255, 0.75)",
            }}
        >
            {description}
        </p>

        {/* Price */}
        <div
            style={{
                fontSize: "24px",
                fontWeight: 700,
                marginTop: "6px",
                color: "#ffffff",
            }}
        >
            {formattedPrice}
        </div>

        {/* Conditional Refundable Badge */}
        {refundable && (
            <div
                style={{
                    fontSize: "13px",
                    display: "flex",
                    alignItems: "center",
                    gap: "6px",
                    color: "rgba(255, 255, 255, 0.9)",
                    marginTop: "4px",
                }}
            >
                <span style={{ color: "#4ade80", fontWeight: "bold" }}>✓</span>
                <span>Refundable</span>
            </div>
        )}
```

```
        </div>
    )
}

// Default values based on your initial object
CourseCard.defaultProps = {
    courseName: "How To YouTube",
    description: "Learn how to build, grow and monetize your channel.",
    mainCategory: "Content Creation",
    pricePaise: 199900,
    refundable: true,
}

// Sidebar controls in Framer
addPropertyControls(CourseCard, {
    courseName: {
        type: ControlType.String,
        title: "Course Name",
        defaultValue: "How To YouTube",
    },
    mainCategory: {
        type: ControlType.String,
        title: "Category",
        defaultValue: "Content Creation",
    },
    description: {
        type: ControlType.String,
        title: "Description",
        defaultValue: "Learn how to build, grow and monetize your channel.",
    },
    pricePaise: {
        type: ControlType.Number,
        title: "Price (Paise)",
        defaultValue: 199900,
        step: 100,
    },
    refundable: {
        type: ControlType.Boolean,
        title: "Refundable",
        defaultValue: true,
    },
})
```

Save the file and check your Framer canvas. When you select the card, you should now see all editable fields in the right sidebar.

Would you like to add interactive hover/animation effects with Framer Motion, or customize the styling (e.g., CTA button, distinct typography)?

User prompt: No Instead of furthur moving with styling, I want to make card dynamic now. means write now we have created one card with hardcoded data, but let's create a card which takes input as a course and then we have list of courses and we map each one with our component. I hope you got it?

Response: Here is how we refactor the code to:

1. Define a TypeScript interface for the `Course` object.
2. Create a reusable `CourseCard` component that accepts a `course` prop.
3. Map over a list (`courses` array) in the main component.

Step 4: Map Over a Dynamic Course List

Replace the content of `CourseCard.tsx` with this updated structure:

```
import * as React from "react"

// 1. Define the Course data structure
export interface Course {
    id: string
    courseName: string
    description: string
    mainCategory: string
    pricePaise: number
    refundable: boolean
}

// 2. Mock list of multiple courses
const COURSES_DATA: Course[] = [
    {
```

```
        id: "course-1",
        courseName: "How To YouTube",
        description: "Learn how to build, grow and monetize your channel.",
        mainCategory: "Content Creation",
        pricePaise: 199900,
        refundable: true,
    },
    {
        id: "course-2",
        courseName: "Framer Masterclass",
        description: "Design and publish responsive websites with custom code.",
        mainCategory: "Web Design",
        pricePaise: 249900,
        refundable: true,
    },
    {
        id: "course-3",
        courseName: "UI/UX Foundations",
        description: "Master typography, color theory, and wireframing from scratch.",
        mainCategory: "Design",
        pricePaise: 149900,
        refundable: false,
    },
]

// 3. Child Component: Single Course Card
export function CourseCard({ course }: { course: Course }) {
    const formattedPrice = new Intl.NumberFormat("en-IN", {
        style: "currency",
        currency: "INR",
        maximumFractionDigits: 0,
    }).format(course.pricePaise / 100)

    return (
        <div
            style={{
                backgroundColor: "#1a1a1a",
                border: "1px dashed rgba(255, 255, 255, 0.3)",
                borderRadius: "12px",
                padding: "24px",
                display: "flex",
                flexDirection: "column",
                gap: "14px",
                color: "#ffffff",
                fontFamily: "Inter, sans-serif",
                width: "280px",
                boxSizing: "border-box",
            }}
        >
            <span
                style={{
                    fontSize: "13px",
                    fontWeight: 500,
                    color: "rgba(255, 255, 255, 0.6)",
                }}
            >
                {course.mainCategory}
            </span>

            <h3
                style={{
                    fontSize: "20px",
                    fontWeight: 700,
                    margin: 0,
                    lineHeight: "1.2",
                }}
            >
                {course.courseName}
            </h3>

            <p
                style={{
                    fontSize: "14px",
                    lineHeight: "1.5",
                    margin: 0,
                    color: "rgba(255, 255, 255, 0.75)",
                }}
            >
                {course.description}
```

```
            </p>

            <div
                style={{
                    fontSize: "22px",
                    fontWeight: 700,
                    marginTop: "auto",
                    paddingTop: "8px",
                }}
            >
                {formattedPrice}
            </div>

            {course.refundable && (
                <div
                    style={{
                        fontSize: "13px",
                        display: "flex",
                        alignItems: "center",
                        gap: "6px",
                        color: "rgba(255, 255, 255, 0.9)",
                    }}
                >
                    <span style={{ color: "#4ade80", fontWeight: "bold" }}>✓</span>
                    <span>Refundable</span>
                </div>
            )}
        </div>
    )
}

// 4. Parent Component: Dynamic List / Grid
export default function CourseList() {
    return (
        <div
            style={{
                display: "flex",
                flexWrap: "wrap",
                gap: "24px",
                justifyContent: "flex-start",
                padding: "20px",
                width: "100%",
            }}
        >
            {COURSES_DATA.map((course) => (
                <CourseCard key={course.id} course={course} />
            ))}
        </div>
    )
}
```

What Changed:

- **`Course` Interface**: Establishes a standard contract for any course object.
- **`CourseCard`**: Accepts a single `course` prop, keeping each card self-contained and modular.
- **`CourseList`**: Maps through `COURSES_DATA` and generates a card for each item with a unique `key`.

Would you like to connect this list to an external API/backend endpoint via `useEffect`/`fetch`, or connect it to Framer's CMS collections next?

User prompt: Something might be wrong with Price converter as I am getting error "Error in CourseCard.tsx Cannot read properties of undefined (reading 'pricePaise')" What is root cuase?

Response: The Root Cause

When Framer renders a component on the canvas or previews it from the sidebar, it instantiates it without passing any props (it passes an empty object `{}`).

Because `course` is `undefined` at initialization, evaluating `course.pricePaise` immediately throws:

```
Cannot read properties of undefined (reading 'pricePaise')
```

The Fix

To resolve this, provide a **default fallback object** and use **optional chaining / nullish coalescing** (`course?.pricePaise ?? 0`) so Framer always has valid data to display on the canvas.

Here is the corrected code:

```
import * as React from "react"

export interface Course {
    id: string
    courseName: string
    description: string
    mainCategory: string
    pricePaise: number
    refundable: boolean
}

// Fallback course data to prevent canvas rendering errors
const DEFAULT_COURSE: Course = {
    id: "default-course",
    courseName: "How To YouTube",
    description: "Learn how to build, grow and monetize your channel.",
    mainCategory: "Content Creation",
    pricePaise: 199900,
    refundable: true,
}

const COURSES_DATA: Course[] = [
    {
        id: "course-1",
        courseName: "How To YouTube",
        description: "Learn how to build, grow and monetize your channel.",
        mainCategory: "Content Creation",
        pricePaise: 199900,
        refundable: true,
    },
    {
        id: "course-2",
        courseName: "Framer Masterclass",
        description: "Design and publish responsive websites with custom code.",
        mainCategory: "Web Design",
        pricePaise: 249900,
        refundable: true,
    },
    {
        id: "course-3",
        courseName: "UI/UX Foundations",
        description: "Master typography, color theory, and wireframing from scratch.",
        mainCategory: "Design",
        pricePaise: 149900,
        refundable: false,
    },
]

// 1. Single Card Component with safe defaults
export function CourseCard({ course = DEFAULT_COURSE }: { course?: Course }) {
    const rawPrice = course?.pricePaise ?? 0

    const formattedPrice = new Intl.NumberFormat("en-IN", {
        style: "currency",
        currency: "INR",
        maximumFractionDigits: 0,
    }).format(rawPrice / 100)

    return (
        <div
            style={{
                backgroundColor: "#1a1a1a",
                border: "1px dashed rgba(255, 255, 255, 0.3)",
                borderRadius: "12px",
                padding: "24px",
                display: "flex",
                flexDirection: "column",
                gap: "14px",
                color: "#ffffff",
                fontFamily: "Inter, sans-serif",
                width: "280px",
                boxSizing: "border-box",
            }}
        >
            <span
                style={{
                    fontSize: "13px",
```

```
                        fontWeight: 500,
                        color: "rgba(255, 255, 255, 0.6)",
                    }}
                >
                    {course?.mainCategory ?? "Category"}
                </span>

                <h3
                    style={{
                        fontSize: "20px",
                        fontWeight: 700,
                        margin: 0,
                        lineHeight: "1.2",
                    }}
                >
                    {course?.courseName ?? "Course Name"}
                </h3>

                <p
                    style={{
                        fontSize: "14px",
                        lineHeight: "1.5",
                        margin: 0,
                        color: "rgba(255, 255, 255, 0.75)",
                    }}
                >
                    {course?.description ?? "No description available."}
                </p>

                <div
                    style={{
                        fontSize: "22px",
                        fontWeight: 700,
                        marginTop: "auto",
                        paddingTop: "8px",
                    }}
                >
                    {formattedPrice}
                </div>

                {course?.refundable && (
                    <div
                        style={{
                            fontSize: "13px",
                            display: "flex",
                            alignItems: "center",
                            gap: "6px",
                            color: "rgba(255, 255, 255, 0.9)",
                        }}
                    >
                        <span style={{ color: "#4ade80", fontWeight: "bold" }}>✓</span>
                        <span>Refundable</span>
                    </div>
                )}
            </div>
        )
    }

    // 2. Parent List Component
    export default function CourseList() {
        return (
            <div
                style={{
                    display: "flex",
                    flexWrap: "wrap",
                    gap: "24px",
                    justifyContent: "flex-start",
                    padding: "20px",
                    width: "100%",
                }}
            >
                {COURSES_DATA.map((item) => (
                    <CourseCard key={item.id} course={item} />
                ))}
            </div>
        )
    }
```

User prompt: Let's move to next lession - can you guide me how we fetch data from API directly? No need to hardcoded it. Here is the reponse schema : { "courseName": "Instagram Growth Lab", "courseCode": "instagram-growth-lab", "description": "Master Reels, carousels, and community building with a repeatable content calendar built for creators who want consistent reach.", "mainCategory": "Social Media", "shortCourse": "Instagram", "courseType": "Original", "pricePaise": 149900, "priceUsdCents": 2999, "mangoId": "b2c3d4e5f67890123456789a", "refundable": true },

Response: Step 5: Fetching Data from an API

To make this completely dynamic:

1. **Update the TypeScript interface** to match your exact API schema.
2. Use **React hooks (`useState`, `useEffect`)** to fetch the data when the component mounts.
3. Handle **Loading**, **Error**, and **Empty** states gracefully.
4. Expose the **API Endpoint URL** as a Framer Property Control so you can change it right from the Framer sidebar.

Updated Code (`CourseCard.tsx`)

Replace the code in your Framer file with the following:

```
import * as React from "react"
import { useState, useEffect } from "react"
import { addPropertyControls, ControlType } from "framer"

// 1. Updated interface based on your API schema
export interface Course {
    courseName: string
    courseCode: string
    description: string
    mainCategory: string
    shortCourse?: string
    courseType?: string
    pricePaise: number
    priceUsdCents?: number
    mangoId?: string
    refundable: boolean
}

// 2. Individual Card Component
export function CourseCard({ course }: { course: Course }) {
    const rawPrice = course?.pricePaise ?? 0

    const formattedPrice = new Intl.NumberFormat("en-IN", {
        style: "currency",
        currency: "INR",
        maximumFractionDigits: 0,
    }).format(rawPrice / 100)

    return (
        <div
            style={{
                backgroundColor: "#1a1a1a",
                border: "1px dashed rgba(255, 255, 255, 0.3)",
                borderRadius: "12px",
                padding: "24px",
                display: "flex",
                flexDirection: "column",
                gap: "14px",
                color: "#ffffff",
                fontFamily: "Inter, sans-serif",
                width: "300px",
                boxSizing: "border-box",
            }}
        >
            <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center" }}>
                <span
                    style={{
                        fontSize: "12px",
                        fontWeight: 500,
                        textTransform: "uppercase",
                        letterSpacing: "0.05em",
                        color: "rgba(255, 255, 255, 0.6)",
                    }}
                >
                    {course.mainCategory || "General"}
                </span>
                {course.courseType && (
                    <span
```

```
                            style={{
                                fontSize: "11px",
                                backgroundColor: "rgba(255, 255, 255, 0.1)",
                                padding: "2px 8px",
                                borderRadius: "999px",
                                color: "#ffffff",
                            }}
                        >
                            {course.courseType}
                        </span>
                    )}
                </div>

                <h3
                    style={{
                        fontSize: "20px",
                        fontWeight: 700,
                        margin: 0,
                        lineHeight: "1.2",
                    }}
                >
                    {course.courseName}
                </h3>

                <p
                    style={{
                        fontSize: "14px",
                        lineHeight: "1.5",
                        margin: 0,
                        color: "rgba(255, 255, 255, 0.75)",
                        flexGrow: 1,
                    }}
                >
                    {course.description}
                </p>

                <div
                    style={{
                        fontSize: "22px",
                        fontWeight: 700,
                        marginTop: "12px",
                    }}
                >
                    {formattedPrice}
                </div>

                {course.refundable && (
                    <div
                        style={{
                            fontSize: "13px",
                            display: "flex",
                            alignItems: "center",
                            gap: "6px",
                            color: "rgba(255, 255, 255, 0.9)",
                        }}
                    >
                        <span style={{ color: "#4ade80", fontWeight: "bold" }}>✓</span>
                        <span>Refundable</span>
                    </div>
                )}
            </div>
        )
    }

    // 3. Main Container Component that handles API fetching
    interface CourseListProps {
        apiUrl: string
    }

    export default function CourseList({ apiUrl }: CourseListProps) {
        const [courses, setCourses] = useState<Course[]>([])
        const [loading, setLoading] = useState<boolean>(true)
        const [error, setError] = useState<string | null>(null)

        useEffect(() => {
            if (!apiUrl) {
                setLoading(false)
                return
            }
```

```
            const fetchCourses = async () => {
                setLoading(true)
                setError(null)

                try {
                    const response = await fetch(apiUrl)
                    if (!response.ok) {
                        throw new Error(`HTTP error! status: ${response.status}`)
                    }
                    const data = await response.json()

                    // Normalize: Handle both array responses and single object responses
                    if (Array.isArray(data)) {
                        setCourses(data)
                    } else if (data && typeof data === "object") {
                        // In case the API returns { courses: [...] } or a single course object
                        const list = data.courses || data.data || [data]
                        setCourses(Array.isArray(list) ? list : [data])
                    }
                } catch (err: any) {
                    setError(err.message || "Failed to fetch courses.")
                } finally {
                    setLoading(false)
                }
            }

            fetchCourses()
        }, [apiUrl])

        // Loading State
        if (loading) {
            return (
                <div style={{ color: "#ffffff", padding: "24px", fontFamily: "Inter, sans-serif" }}>
                    Loading courses...
                </div>
            )
        }

        // Error State
        if (error) {
            return (
                <div style={{ color: "#ff6b6b", padding: "24px", fontFamily: "Inter, sans-serif" }}>
                    Error: {error}
                </div>
            )
        }

        // Empty State
        if (courses.length === 0) {
            return (
                <div style={{ color: "rgba(255, 255, 255, 0.5)", padding: "24px", fontFamily: "Inter, sans-serif" }}>
                    No courses found. Enter a valid API URL in the sidebar.
                </div>
            )
        }

        // Dynamic Render
        return (
            <div
                style={{
                    display: "flex",
                    flexWrap: "wrap",
                    gap: "24px",
                    justifyContent: "flex-start",
                    padding: "20px",
                    width: "100%",
                }}
            >
                {courses.map((item) => (
                    <CourseCard
                        key={item.mangoId || item.courseCode || item.courseName}
                        course={item}
                    />
                ))}
            </div>
        )
    }
```

```
// Default Props
CourseList.defaultProps = {
    apiUrl: "https://api.example.com/courses", // Replace with your real endpoint
}

// Property Controls for Framer Sidebar
addPropertyControls(CourseList, {
    apiUrl: {
        type: ControlType.String,
        title: "API URL",
        placeholder: "https://your-api.com/courses",
    },
})
```

Key Additions:

- **Flexible Payload Handling:** Works whether your API returns a single course object, an array of courses `[...]`, or a wrapped object `{ courses: [...] }`.
- **Unique Key Selection:** Uses `mangoId` or `courseCode` from your response schema as the React `key`.
- **Sidebar URL Control:** Paste your live backend URL directly into the Framer property panel.

User prompt: What is the Possible root cause on getting error while 'Error in CourseCard.tsx Cannot read properties of undefined (reading 'mainCategory')'

Response: The error **`Cannot read properties of undefined (reading 'mainCategory')`** means that the **`course`** **object itself is `undefined`** at runtime when JavaScript tries to access `course.mainCategory`.

Here are the most common root causes in Framer and how to resolve them:

1. Direct Canvas Rendering (Most Likely)

When you export `export function CourseCard(...)` alongside `CourseList`, Framer's internal renderer inspects all exported React components in the file and tries to mount them independently in the canvas without passing any props.

- **Why it breaks:** Framer renders `<CourseCard/>` with `props = {}`, making `course = undefined`.
- In the previous step, code like `{course.mainCategory}` was accessed without safe navigation.

2. Missing Optional Chaining (`?.`)

If `course` is `undefined` or temporarily missing during render:

- `course.mainCategory` ❌ **Crashes**
- `course?.mainCategory` **Safely returns `undefined`** instead of throwing an error.

3. API Returned `null` or Empty Elements in Array

If the API returns an array containing `null`, `undefined`, or empty entries (e.g., `[null, {...}]`), `.map()` will pass `undefined` into `<CourseCard course="{item}"/>`.

4. API Response Structure Mismatch

If the API returns an object wrapped differently (e.g., `{ result: { ... } }` or `{ error: "Unauthorized" }`), the code might map over invalid objects.

The Fix

Update your `CourseCard` child component to include a **fallback default object** and **optional chaining (`?.`)**:

```
// 1. Fallback placeholder
const DEFAULT_COURSE: Course = {
    courseName: "Sample Course",
    courseCode: "sample-course",
    description: "Course description placeholder.",
    mainCategory: "General",
    pricePaise: 0,
    refundable: false,
}

// 2. Safe Component Definition
export function CourseCard({ course = DEFAULT_COURSE }: { course?: Course }) {
    // Safely fallback if course or course.pricePaise is missing
    const rawPrice = course?.pricePaise ?? 0

    const formattedPrice = new Intl.NumberFormat("en-IN", {
```

```
            style: "currency",
            currency: "INR",
            maximumFractionDigits: 0,
        }).format(rawPrice / 100)

        return (
            <div
                style={{
                    backgroundColor: "#1a1a1a",
                    border: "1px dashed rgba(255, 255, 255, 0.3)",
                    borderRadius: "12px",
                    padding: "24px",
                    display: "flex",
                    flexDirection: "column",
                    gap: "14px",
                    color: "#ffffff",
                    fontFamily: "Inter, sans-serif",
                    width: "300px",
                    boxSizing: "border-box",
                }}
            >
                <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center" }}>
                    <span
                        style={{
                            fontSize: "12px",
                            fontWeight: 500,
                            textTransform: "uppercase",
                            letterSpacing: "0.05em",
                            color: "rgba(255, 255, 255, 0.6)",
                        }}
                    >
                        {course?.mainCategory || "General"}
                    </span>
                    {course?.courseType && (
                        <span
                            style={{
                                fontSize: "11px",
                                backgroundColor: "rgba(255, 255, 255, 0.1)",
                                padding: "2px 8px",
                                borderRadius: "999px",
                                color: "#ffffff",
                            }}
                        >
                            {course.courseType}
                        </span>
                    )}
                </div>

                <h3
                    style={{
                        fontSize: "20px",
                        fontWeight: 700,
                        margin: 0,
                        lineHeight: "1.2",
                    }}
                >
                    {course?.courseName || "Untitled Course"}
                </h3>

                <p
                    style={{
                        fontSize: "14px",
                        lineHeight: "1.5",
                        margin: 0,
                        color: "rgba(255, 255, 255, 0.75)",
                        flexGrow: 1,
                    }}
                >
                    {course?.description || "No description provided."}
                </p>

                <div
                    style={{
                        fontSize: "22px",
                        fontWeight: 700,
                        marginTop: "12px",
                    }}
                >
                    {formattedPrice}
```

```
            </div>

            {course?.refundable && (
                <div
                    style={{
                        fontSize: "13px",
                        display: "flex",
                        alignItems: "center",
                        gap: "6px",
                        color: "rgba(255, 255, 255, 0.9)",
                    }}
                >
                    <span style={{ color: "#4ade80", fontWeight: "bold" }}>✓</span>
                    <span>Refundable</span>
                </div>
            )}
        </div>
    )
}
```

User prompt: Hey again First explain what we are going to write and then give me a code please

Response: Here is a clear breakdown of what we are building, followed by the complete, ready-to-use code.

What We Are Going to Write

1. **TypeScript Interface (`Course`)**
   - Mirrors your exact API payload schema (`courseName`, `courseCode`, `description`, `mainCategory`, `pricePaise`, `refundable`, etc.).
2. **Default Fallback Data (`DEFAULT_COURSE`)**
   - Protects Framer from crashing. When Framer mounts or inspects an exported component in the canvas sidebar, it doesn't pass props. Having a safe fallback ensures it never reads from an `undefined` object.
3. **Defensive Child Component (`CourseCard`)**
   - Uses default props (`course = DEFAULT_COURSE`) and optional chaining (`course?.mainCategory`, `course?.pricePaise ?? 0`).
   - Converts the raw paise price to formatted Indian Rupees (₹) dynamically.
   - Renders the card layout with category tags, title, description, price, and refundable status.
4. **Parent Component with API Lifecycle (`CourseList`)**
   - Uses `useState` to track three essential states: `courses` (data), `loading` (fetching in progress), and `error` (network/server issues).
   - Uses `useEffect` to fetch data whenever the `apiUrl` property changes.
   - Normalizes the API response so it works whether your backend returns a direct array (`[...]`), a single course object (`{...}`), or a wrapped object (`{ data: [...] }`).
5. **Framer Property Controls (`addPropertyControls`)**
   - Exposes an editable `API URL` field directly in the Framer right sidebar.

Complete Code (`CourseCard.tsx`)

Replace the entire contents of your file with this:

```
import * as React from "react"
import { useState, useEffect } from "react"
import { addPropertyControls, ControlType } from "framer"

// 1. TypeScript Interface matching your API schema
export interface Course {
    courseName: string
    courseCode: string
    description: string
    mainCategory: string
    shortCourse?: string
    courseType?: string
    pricePaise: number
    priceUsdCents?: number
    mangoId?: string
    refundable: boolean
}

// 2. Safe default placeholder to prevent Framer canvas inspect errors
const DEFAULT_COURSE: Course = {
```

```
        courseName: "Instagram Growth Lab",
        courseCode: "instagram-growth-lab",
        description:
            "Master Reels, carousels, and community building with a repeatable content calendar built for creators who w
        mainCategory: "Social Media",
        shortCourse: "Instagram",
        courseType: "Original",
        pricePaise: 149900,
        priceUsdCents: 2999,
        mangoId: "b2c3d4e5f67890123456789a",
        refundable: true,
    }

    // 3. Child Card Component (Defensively coded)
    export function CourseCard({ course = DEFAULT_COURSE }: { course?: Course }) {
        const rawPrice = course?.pricePaise ?? 0

        const formattedPrice = new Intl.NumberFormat("en-IN", {
            style: "currency",
            currency: "INR",
            maximumFractionDigits: 0,
        }).format(rawPrice / 100)

        return (
            <div
                style={{
                    backgroundColor: "#1a1a1a",
                    border: "1px dashed rgba(255, 255, 255, 0.3)",
                    borderRadius: "12px",
                    padding: "24px",
                    display: "flex",
                    flexDirection: "column",
                    gap: "14px",
                    color: "#ffffff",
                    fontFamily: "Inter, sans-serif",
                    width: "300px",
                    boxSizing: "border-box",
                }}
            >
                {/* Header: Category & Badge */}
                <div
                    style={{
                        display: "flex",
                        justifyContent: "space-between",
                        alignItems: "center",
                    }}
                >
                    <span
                        style={{
                            fontSize: "12px",
                            fontWeight: 500,
                            textTransform: "uppercase",
                            letterSpacing: "0.05em",
                            color: "rgba(255, 255, 255, 0.6)",
                        }}
                    >
                        {course?.mainCategory || "General"}
                    </span>

                    {course?.courseType && (
                        <span
                            style={{
                                fontSize: "11px",
                                backgroundColor: "rgba(255, 255, 255, 0.12)",
                                padding: "2px 8px",
                                borderRadius: "999px",
                                color: "#ffffff",
                            }}
                        >
                            {course.courseType}
                        </span>
                    )}
                </div>

                {/* Course Title */}
                <h3
                    style={{
                        fontSize: "20px",
                        fontWeight: 700,
```

```
                        margin: 0,
                        lineHeight: "1.2",
                    }}
                >
                    {course?.courseName || "Untitled Course"}
                </h3>

                {/* Description */}
                <p
                    style={{
                        fontSize: "14px",
                        lineHeight: "1.5",
                        margin: 0,
                        color: "rgba(255, 255, 255, 0.75)",
                        flexGrow: 1,
                    }}
                >
                    {course?.description || "No description provided."}
                </p>

                {/* Price */}
                <div
                    style={{
                        fontSize: "22px",
                        fontWeight: 700,
                        marginTop: "12px",
                    }}
                >
                    {formattedPrice}
                </div>

                {/* Refundable Badge */}
                {course?.refundable && (
                    <div
                        style={{
                            fontSize: "13px",
                            display: "flex",
                            alignItems: "center",
                            gap: "6px",
                            color: "rgba(255, 255, 255, 0.9)",
                        }}
                    >
                        <span style={{ color: "#4ade80", fontWeight: "bold" }}>✓</span>
                        <span>Refundable</span>
                    </div>
                )}
            </div>
        )
    }

    // 4. Parent Component (Data Fetching & State)
    interface CourseListProps {
        apiUrl: string
    }

    export default function CourseList({ apiUrl }: CourseListProps) {
        const [courses, setCourses] = useState<Course[]>([])
        const [loading, setLoading] = useState<boolean>(true)
        const [error, setError] = useState<string | null>(null)

        useEffect(() => {
            if (!apiUrl) {
                setCourses([DEFAULT_COURSE])
                setLoading(false)
                return
            }

            const fetchCourses = async () => {
                setLoading(true)
                setError(null)

                try {
                    const response = await fetch(apiUrl)
                    if (!response.ok) {
                        throw new Error(`Server returned HTTP ${response.status}`)
                    }
                    const data = await response.json()

                    // Normalize array or wrapped object responses
```

```
            if (Array.isArray(data)) {
                setCourses(data)
            } else if (data && typeof data === "object") {
                const list = data.courses || data.data || [data]
                setCourses(Array.isArray(list) ? list : [data])
            } else {
                setCourses([DEFAULT_COURSE])
            }
        } catch (err: any) {
            setError(err.message || "Failed to fetch courses.")
        } finally {
            setLoading(false)
        }
    }

    fetchCourses()
}, [apiUrl])

// Loading State
if (loading) {
    return (
        <div
            style={{
                color: "rgba(255, 255, 255, 0.7)",
                padding: "24px",
                fontFamily: "Inter, sans-serif",
            }}
        >
            Loading courses...
        </div>
    )
}

// Error State
if (error) {
    return (
        <div
            style={{
                color: "#ff6b6b",
                padding: "24px",
                fontFamily: "Inter, sans-serif",
            }}
        >
            Error: {error}
        </div>
    )
}

// Empty State
if (courses.length === 0) {
    return (
        <div
            style={{
                color: "rgba(255, 255, 255, 0.5)",
                padding: "24px",
                fontFamily: "Inter, sans-serif",
            }}
        >
            No courses available.
        </div>
    )
}

// Render Grid
return (
    <div
        style={{
            display: "flex",
            flexWrap: "wrap",
            gap: "24px",
            justifyContent: "flex-start",
            padding: "20px",
            width: "100%",
            boxSizing: "border-box",
        }}
    >
        {courses.map((item, index) => (
            <CourseCard
                key={item?.mangoId || item?.courseCode || `course-${index}`}
```

```
                    course={item}
                />
            ))}
        </div>
    )
}

// 5. Default Props & Property Controls
CourseList.defaultProps = {
    apiUrl: "",
}

addPropertyControls(CourseList, {
    apiUrl: {
        type: ControlType.String,
        title: "API URL",
        placeholder: "https://your-api-endpoint.com/courses",
    },
})
```

---

*User prompt: import * as React from "react" import { useState, useEffect } from "react" import { addPropertyControls, ControlType } from "framer" // 1. TypeScript Interface matching your API schema export interface Course { courseName: string courseCode: string description: string mainCategory: string shortCourse?: string courseType?: string pricePaise: number priceUsdCents?: number mangoId?: string refundable: boolean } // 2. Safe default placeholder to prevent Framer canvas inspect errors const DEFAULT_COURSE: Course = { courseName: "Instagram Growth Lab", courseCode: "instagram-growth-lab", description: "Master Reels, carousels, and community building with a repeatable content calendar built for creators who want consistent reach.", mainCategory: "Social Media", shortCourse: "Instagram", courseType: "Original", pricePaise: 149900, priceUsdCents: 2999, mangoId: "b2c3d4e5f67890123456789a", refundable: true, } // 3. Child Card Component (Defensively coded) export function CourseCard({ course = DEFAULT_COURSE }: { course?: Course }) { const rawPrice = course?.pricePaise ?? 0 const formattedPrice = new Intl.NumberFormat("en-IN", { style: "currency", currency: "INR", maximumFractionDigits: 0, }).format(rawPrice / 100) return (<div style={{ backgroundColor: "#1a1a1a", border: "1px dashed rgba(255, 255, 255, 0.3)", borderRadius: "12px", padding: "24px", display: "flex", flexDirection: "column", gap: "14px", color: "#ffffff", fontFamily: "Inter, sans-serif", width: "300px", boxSizing: "border-box", }} > {/* Header: Category & Badge */} <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", }} > <span style={{ fontSize: "12px", fontWeight: 500, textTransform: "uppercase", letterSpacing: "0.05em", color: "rgba(255, 255, 255, 0.6)", }} > {course?.mainCategory || "General"} {course?.courseType && (<span style={{ fontSize: "11px", backgroundColor: "rgba(255, 255, 255, 0.12)", padding: "2px 8px", borderRadius: "999px", color: "#ffffff", }} > {course.courseType})} {/* Course Title */} <h3 style={{ fontSize: "20px", fontWeight: 700, margin: 0, lineHeight: "1.2", }} > {course?.courseName || "Untitled Course"} </h3> {/* Description */} <p style={{ fontSize: "14px", lineHeight: "1.5", margin: 0, color: "rgba(255, 255, 255, 0.75)", flexGrow: 1, }} > {course?.description || "No description provided."} </p> {/* Price */} <div style={{ fontSize: "22px", fontWeight: 700, marginTop: "12px", }} > {formattedPrice} {/* Refundable Badge */} {course?.refundable && (<div style={{ fontSize: "13px", display: "flex", alignItems: "center", gap: "6px", color: "rgba(255, 255, 255, 0.9)", }} > <span style={{ color: "#4ade80", fontWeight: "bold" }}> ✓ Refundable)}) } // 4. Parent Component (Data Fetching & State) interface CourseListProps { apiUrl: string } export default function CourseList({ apiUrl }: CourseListProps) { const [courses, setCourses] = useState<Course[]>([]) const [loading, setLoading] = useState<boolean>(true) const [error, setError] = useState<string | null>(null) useEffect(() => { if (!apiUrl) { setCourses([DEFAULT_COURSE]) setLoading(false) return } const fetchCourses = async () => { setLoading(true) setError(null) try { const response = await fetch(apiUrl) if (!response.ok) { throw new Error(`Server returned HTTP ${response.status}`) } const data = await response.json() // Normalize array or wrapped object responses if (Array.isArray(data)) { setCourses(data) } else if (data && typeof data === "object") { const list = data.courses || data.data || [data] setCourses(Array.isArray(list) ? list : [data]) } else { setCourses([DEFAULT_COURSE]) } } catch (err: any) { setError(err.message || "Failed to fetch courses.") } finally { setLoading(false) } } fetchCourses() }, [apiUrl]) // Loading State if (loading) { return (<div style={{ color: "rgba(255, 255, 255, 0.7)", padding: "24px", fontFamily: "Inter, sans-serif", }} > Loading courses...) } // Error State if (error) { return (<div style={{ color: "#ff6b6b", padding: "24px", fontFamily: "Inter, sans-serif", }} > Error: {error}) } // Empty State if (courses.length === 0) { return (<div style={{ color: "rgba(255, 255, 255, 0.5)", padding: "24px", fontFamily: "Inter, sans-serif", }} > No courses available.) } // Render Grid return (<div style={{ display: "flex", flexWrap: "wrap", gap: "24px", justifyContent: "flex-start", padding: "20px", width: "100%", boxSizing: "border-box", }} > {courses.map((item, index) => (<CourseCard key={item?.mangoId || item?.courseCode || `course-${index}`} course={item} />))}) } // 5. Default Props & Property Controls CourseList.defaultProps = { apiUrl: "", } addPropertyControls(CourseList, { apiUrl: { type: ControlType.String, title: "API URL", placeholder: "https://syncsphere-hiv6.onrender.com/assignment/course-data", }, }) - so I try to perform call but only getting defaul card one. even I don't have any idea that How can debug in frammer to check network tab or console statement or whatever it handle inside.*

Response: 1. How to Debug in Framer (Console & Network Tab)

Framer is built on web technologies (Chromium / React), so you have access to the exact same Developer Tools as standard web development:

If you are using Framer in the Browser (Chrome, Brave, Edge):

- Press **`F12`** or **`Ctrl + Shift + I`** (Windows) / **`Cmd + Option + I`** (Mac).
- Go to the **Console** tab to see your `console.log` statements and JavaScript errors.

- Go to the **Network** tab, filter by **Fetch/XHR**, and reload or re-render to see the actual HTTP request, status code (200, 404, 500, CORS), and response payload.

If you are using the Framer Desktop App:

- Press **`Ctrl + Shift + I`** (Windows) or **`Cmd + Option + I`** (Mac) directly inside Framer.
- Alternatively, open the **Preview** (play button in top right), click the pop-out/browser icon, and inspect from your browser.

2. Why You Only Saw the Default Card (Root Cause)

1. **`placeholder` is not a value:** In Framer property controls, `placeholder` is only the grey hint text shown in the input box when it is empty. It does not pass any value to your component.
2. **`defaultProps.apiUrl` was set to `""`:** Because `apiUrl` was empty (`""`), the condition `if (!apiUrl)` evaluated to `true`, immediately executing `setCourses([DEFAULT_COURSE])` and exiting before any fetch call could run.

3. Updated Code with Built-In Debugging Logs

Here is the updated code where:

- The real URL is set as both `defaultValue` and `defaultProps`.
- `console.log` statements are added inside `fetchCourses` so you can watch the entire lifecycle directly in your browser/Framer console.

```
import * as React from "react"
import { useState, useEffect } from "react"
import { addPropertyControls, ControlType } from "framer"

// 1. TypeScript Interface matching API schema
export interface Course {
    courseName: string
    courseCode: string
    description: string
    mainCategory: string
    shortCourse?: string
    courseType?: string
    pricePaise: number
    priceUsdCents?: number
    mangoId?: string
    refundable: boolean
}

// 2. Safe default placeholder
const DEFAULT_COURSE: Course = {
    courseName: "Instagram Growth Lab",
    courseCode: "instagram-growth-lab",
    description:
        "Master Reels, carousels, and community building with a repeatable content calendar built for creators who w
    mainCategory: "Social Media",
    shortCourse: "Instagram",
    courseType: "Original",
    pricePaise: 149900,
    priceUsdCents: 2999,
    mangoId: "b2c3d4e5f67890123456789a",
    refundable: true,
}

// 3. Child Card Component
export function CourseCard({ course = DEFAULT_COURSE }: { course?: Course }) {
    const rawPrice = course?.pricePaise ?? 0

    const formattedPrice = new Intl.NumberFormat("en-IN", {
        style: "currency",
        currency: "INR",
        maximumFractionDigits: 0,
    }).format(rawPrice / 100)

    return (
        <div
            style={{
                backgroundColor: "#1a1a1a",
                border: "1px dashed rgba(255, 255, 255, 0.3)",
                borderRadius: "12px",
                padding: "24px",
                display: "flex",
                flexDirection: "column",
                gap: "14px",
                color: "#ffffff",
                fontFamily: "Inter, sans-serif",
```

```
            width: "300px",
            boxSizing: "border-box",
        }}
    >
        {/* Header: Category & Badge */}
        <div
            style={{
                display: "flex",
                justifyContent: "space-between",
                alignItems: "center",
            }}
        >
            <span
                style={{
                    fontSize: "12px",
                    fontWeight: 500,
                    textTransform: "uppercase",
                    letterSpacing: "0.05em",
                    color: "rgba(255, 255, 255, 0.6)",
                }}
            >
                {course?.mainCategory || "General"}
            </span>

            {course?.courseType && (
                <span
                    style={{
                        fontSize: "11px",
                        backgroundColor: "rgba(255, 255, 255, 0.12)",
                        padding: "2px 8px",
                        borderRadius: "999px",
                        color: "#ffffff",
                    }}
                >
                    {course.courseType}
                </span>
            )}
        </div>

        {/* Course Title */}
        <h3
            style={{
                fontSize: "20px",
                fontWeight: 700,
                margin: 0,
                lineHeight: "1.2",
            }}
        >
            {course?.courseName || "Untitled Course"}
        </h3>

        {/* Description */}
        <p
            style={{
                fontSize: "14px",
                lineHeight: "1.5",
                margin: 0,
                color: "rgba(255, 255, 255, 0.75)",
                flexGrow: 1,
            }}
        >
            {course?.description || "No description provided."}
        </p>

        {/* Price */}
        <div
            style={{
                fontSize: "22px",
                fontWeight: 700,
                marginTop: "12px",
            }}
        >
            {formattedPrice}
        </div>

        {/* Refundable Badge */}
        {course?.refundable && (
            <div
                style={{
```

```
                            fontSize: "13px",
                            display: "flex",
                            alignItems: "center",
                            gap: "6px",
                            color: "rgba(255, 255, 255, 0.9)",
                        }}
                    >
                        <span style={{ color: "#4ade80", fontWeight: "bold" }}>
                            ✓
                        </span>
                        <span>Refundable</span>
                    </div>
                )}
            </div>
        )
    }

    // 4. Parent Component with Diagnostics
    interface CourseListProps {
        apiUrl: string
    }

    export default function CourseList({ apiUrl }: CourseListProps) {
        const [courses, setCourses] = useState<Course[]>([])
        const [loading, setLoading] = useState<boolean>(true)
        const [error, setError] = useState<string | null>(null)

        useEffect(() => {
            console.log("[CourseList] Current apiUrl prop:", apiUrl)

            if (!apiUrl || apiUrl.trim() === "") {
                console.warn("[CourseList] No API URL provided. Falling back to default course.")
                setCourses([DEFAULT_COURSE])
                setLoading(false)
                return
            }

            const fetchCourses = async () => {
                setLoading(true)
                setError(null)
                console.log("[CourseList] Initiating fetch to:", apiUrl)

                try {
                    const response = await fetch(apiUrl)
                    console.log("[CourseList] HTTP Status:", response.status)

                    if (!response.ok) {
                        throw new Error(`Server returned HTTP ${response.status}`)
                    }

                    const data = await response.json()
                    console.log("[CourseList] Raw API Response Data:", data)

                    // Normalize data structure
                    if (Array.isArray(data)) {
                        setCourses(data)
                    } else if (data && typeof data === "object") {
                        const list = data.courses || data.data || [data]
                        setCourses(Array.isArray(list) ? list : [data])
                    } else {
                        setCourses([DEFAULT_COURSE])
                    }
                } catch (err: any) {
                    console.error("[CourseList] Fetch error caught:", err)
                    setError(err.message || "Failed to fetch courses.")
                } finally {
                    setLoading(false)
                }
            }

            fetchCourses()
        }, [apiUrl])

        if (loading) {
            return (
                <div
                    style={{
                        color: "rgba(255, 255, 255, 0.7)",
                        padding: "24px",
```

```
                    fontFamily: "Inter, sans-serif",
                }}
            >
                Loading courses from API...
            </div>
        )
    }

    if (error) {
        return (
            <div
                style={{
                    color: "#ff6b6b",
                    padding: "24px",
                    fontFamily: "Inter, sans-serif",
                }}
            >
                <div>Error loading courses: {error}</div>
                <div style={{ fontSize: "12px", marginTop: "8px", opacity: 0.8 }}>
                    Check Developer Tools Console for full details.
                </div>
            </div>
        )
    }

    if (courses.length === 0) {
        return (
            <div
                style={{
                    color: "rgba(255, 255, 255, 0.5)",
                    padding: "24px",
                    fontFamily: "Inter, sans-serif",
                }}
            >
                No courses available.
            </div>
        )
    }

    return (
        <div
            style={{
                display: "flex",
                flexWrap: "wrap",
                gap: "24px",
                justifyContent: "flex-start",
                padding: "20px",
                width: "100%",
                boxSizing: "border-box",
            }}
        >
            {courses.map((item, index) => (
                <CourseCard
                    key={item?.mangoId || item?.courseCode || `course-${index}`}
                    course={item}
                />
            ))}
        </div>
    )
}

// 5. Default Props & Property Controls
CourseList.defaultProps = {
    apiUrl: "https://syncsphere-hiv6.onrender.com/assignment/course-data",
}

addPropertyControls(CourseList, {
    apiUrl: {
        type: ControlType.String,
        title: "API URL",
        defaultValue: "https://syncsphere-hiv6.onrender.com/assignment/course-data",
    },
})
```

What to check after updating:

1. Open the Developer Tools (**`Ctrl + Shift + I`** or **`Cmd + Option + I`**).
2. Look at the **Console** tab for logs starting with `[CourseList]`.

3. Note: If your Render.com server is sleeping (free tier spinning down due to inactivity), the first fetch may take ~30–50 seconds to respond before the cards populate.
